

Submitted To : Ms. Ritika Mam

Submitted By : Gourav

Subject : Data Structures (PC IT 301)

Class : B.E. IT (2nd Year)

Roll No. : UE238037

Write a program to solve Dutch National Flag Problem.

```
// Dutch National Flag Algorithm is an algorithm
// which is used to sort an array of 0's, 1's, 2's
// in a linear time complexity O(n)

// HEADER FILES
#include<iostream>
#include<vector>
using namespace std;

// Function to input the array,
// where vector(array) is being passed as reference
void input(vector<int> &v, int n){
   for(int i = 0; i < n; i++){
      cin >> v[i];
   }
}

// Function to print the array
void printArray(vector<int> &v, int n){
   for(int i = 0; i < n; i++){
      cout << v[i] << " ";
   }cout << "\n";
}

// Dutch National Flag algorithm
void dnfAlgo(vector<int> &v, int n){
   int l = 0, m = 0, h = n - 1;
```

```
    while(m <= h){
        if(v[m] == 0){
            swap(v[l],v[m]);
            l++;
            m++;
        }else if(v[m] == 1){
            m++;
        }else{
            swap(v[m],v[h]);
            h--;
        }
    }
}

// main function(execution begins from here)
int main(){
    int n; // where n is the size of the array
    cout << "Enter the size of the array : ";
    cin >> n;

    vector<int> a(n, 0); // initialise an array of n elements with value 0
    // Input Array
    cout << "Enter the array elements : ";
    input(a,n);

    // Printing array before sorting
    cout << "Array before :\n";
    printArray(a,n);
```

```
    // Sorting
    cout << "Sorting the array using Dutch's National Flag Algorithm....\n";
    dnfAlgo(a,n);

    // Printing array after sorting
    cout << "Array after :\n";
    printArray(a,n);

    return 0; // indicates no error
}
```

OUTPUT

```
PS C:\Users\junej\CPP> cd "c:\Users\junej\CPP\" ; if ($?) { g++ DutchNationalFlag.cpp -o DutchNationalFlag } ; if ($?) { .\
DutchNationalFlag }
Enter the size of the array : 7
Enter the array elements : 2 1 0 0 2 1 1
Array before :
2 1 0 0 2 1 1
Sorting the array using Dutch's National Flag Algorithm....
Array after :
0 0 1 1 1 2 2
PS C:\Users\junej\CPP>
```

Write a program to solve Maximum Sum Subarray Problem (Kadane's Algorithm).

```
// Kadane's algorithm is one which is used to calculate
// the maximum subarray sum in a given array, where the
// subarray is a contigous part of an array

// HEADER FILES
#include<iostream>
#include<vector>
using namespace std;

// Function to input the array, where vector(array) is passed by reference
void input(vector<int> &v, int n){
   for(int i=0; i<n; i++){
      cin >> v[i];
   }
}

// Function to calculate MAXIMUM SUBARRAY SUM
void maxSubarraySum(vector<int> &v, int n){
   int currSum = 0, maxSum = INT_MIN;
   for(int i=0; i<n; i++){
      currSum += v[i];
      if(currSum > maxSum){
         maxSum = currSum;
      }
      if(currSum < 0){
         currSum = 0;
```

```
        }
    }
    // Output Maximum Subarray Sum
    cout << "The max Subarray sum calculated by Kadanne's Algorithm is : " << maxSum << "\n";
}

// main function(execution begins from here)
int main(){
    int n; // where n is the size of the array
    cout << "Enter the number of elements : ";
    cin >> n;

    vector<int> v(n, 0); // Creating & Initialising an array of n elements with value 0
    // Input array elements
    cout << "Enter the elements of the array : ";
    input(v, n);

    // Function Call to print the maximum subarray Sum
    maxSubarraySum(v, n);

    return 0; // returning 0 indicated no error
}
```

OUTPUT

```
PS C:\Users\junej\CPP> cd "c:\Users\junej\CPP\" ; if ($?) { g++ KadaneSalgorithm.cpp -o KadaneSalgorithm } ; if ($?) { .\Ka
daneSalgorithm }
Enter the number of elements : 5
Enter the elements of the array : 1 2 3 -2 5
The max Subarray sum calculated by Kadanne's Algorithm is : 9
```

Write a program to solve Boyer–Moore Majority Vote Algorithm.

```
// Boyer Moore's Majority Vote Algorithm is used is used
// to calculate the Majority Element present in the array
// By majority element we means the element which is present
// (> n/2) in the array i.e more than a half.

// HEADER FILES
#include<iostream>
#include<vector>
using namespace std;

// Function to Input the array, where vector(array) is passed as reference
void input(vector<int> &v, int n){
   for(int i=0; i<n; i++)cin >> v[i];
}

// Function to find the majority element in the array(which exists for > n/2 times)
int MajorityElement(vector<int> &v, int n){
   int freq = 0, ans = v[0];
   for(int i=0; i < n; i++){
      if(freq == 0)ans = v[i];
      if(ans == v[i])freq++;
      else freq--;
   }
   return ans;
}

// main function(execution begins from here)
```

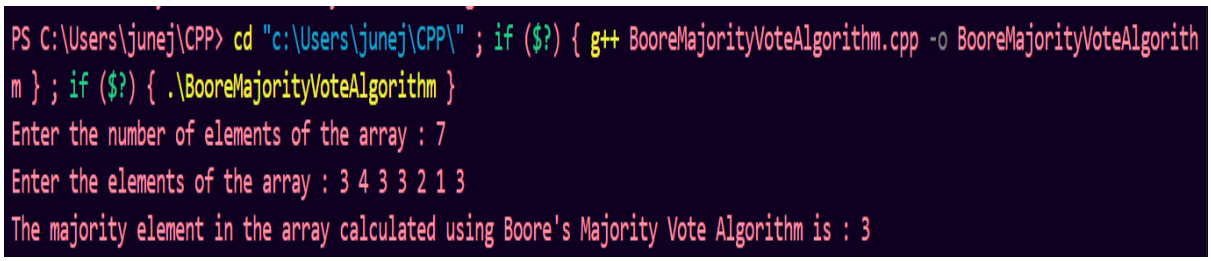
```
int main(){
    int n; // where n is the size of the array
    cout << "Enter the number of elements of the array : ";
    cin >> n;

    vector<int> v(n,0); // initialising an array of n elements with value 0
    // Input array elements
    cout << "Enter the elements of the array : ";
    input(v, n);

    // Printing majority element
    cout << "The majority element in the array calculated using Boore's Majority Vote Algorithm is : "<<MajorityElement(v,n) << "\n";

    return 0; // Indicates no error
}
```

OUTPUT



Write a program to solve Word Break Problem .

```
// Word  Break is a problem in which  the goal is to  determine
// if a  given string can  be augmented  into space - separated
// sequence of one or more dictionary words. Given a dictionary
// (list of words), we have to return true if the string can be
// segmented, false otherwise

// HEADER FILES
#include<iostream>
#include<vector>
#include<string>
#include<unordered_set>
using namespace std;

// Function to check if word break is possible or not
bool wordBreak(string &target, vector<string> wordList){
   // converting given dictionary of valid words into an unordered set
   // so as to make the time of looks ups of words of O(1) average-time
   unordered_set<string> validWords(wordList.begin(), wordList.end());

   // length of the given string
   int n = target.length();
   // vector check to keep check of the sub string
   vector<bool> check(n + 1, false);
   check[0] = true; // because an empty string can always be formed from given words

   // To check for complete string
   for(int i = 1; i <= n; i++){
```

```
        for(int j = 0; j < i; j++){
            if(check[j] && validWords.find(target.substr(j, i-j)) != validWords.end()){
                check[i] = true;
                break; // exits inner loop if substr is found
            }
        }
    }
    return check[n];
}

int main(){
    string s;
    // Input string to be augmented(broken into smaller words)
    cout << "Enter the string : ";
    getline(cin,s);

    int n;
    // Input dictionary size
    cout << "Enter the number of words in the dictionary : ";
    cin >> n;

    vector<string> wordList(n);
    // where word list is the list of valid words
    // which should match with the augmented string

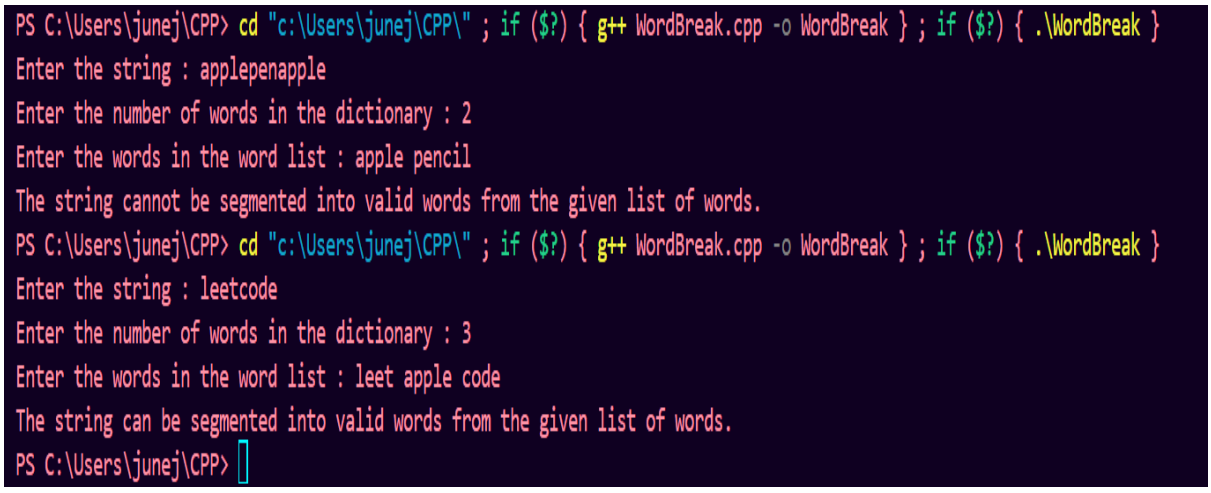
    // Input the words in dictionary
    cout << "Enter the words in the word list : ";
    for(int i = 0; i < n; i++){
        cin >> wordList[i];
```

```
    }

    // Check if the string can be segmented using
    // FUNCTION CALL inside CONDITIONAL STATEMENT
    if(wordBreak(s, wordList)){
        cout << "The string can be segmented into valid words from the given list of words.";
    }else{
        cout << "The string cannot be segmented into valid words from the given list of words.";
    }cout << "\n"; // next-line

    return 0; // indicates no error
}
```

OUTPUT



Write a program to implement stack using linked list .

```
// Implement Stack using Linked List

// HEADER FILES
#include<iostream>
using namespace std;

// Creating a class Node
class Node{
public:
   int data; // data indicates value
   Node* next; // next indicates next pointer

   // CONSTRUCTOR
   Node(int data){
      this->data = data;
      next = nullptr;
   }
};

// Creating Stack Class using concept of Linked list
class Stack{
private:
   Node* top;

public:
   // CONSTRUCTOR
   Stack(){
```

```
    top = nullptr;
  }

  // push function to input some value in the data
  void push(int data){
    Node* newNode = new Node(data);
    if(!newNode){
      // If new Node is not created it means Heap Memory is Full(overflow)
      cout << "Heap Overflow\n";
      return;
    }
    newNode->next = top;
    top = newNode;
    cout << data << " pushed onto stack\n";
  }

  // pop function to remove data from the stack
  void pop(){
    // if top points to the null pointer that means no element exists in the stack or stack is
empty
    if(top == nullptr){
      cout << "Stack underflow\n";
      return;
    }
    // else
    Node* temp = top;
    top = top->next;
    cout << temp->data << " popped from stack\n";
    delete temp;
  }
```

```
// returns topmost element if stack is not empty else display a note
int peek(){
    if(top != nullptr){
        return top->data;
    }else{
        cout << "Stack is empty\n";
        return -1;
    }
}

// checks if the stack is empty or not
bool isEmpty(){
    return top == nullptr;
}

// prints all elements of the stack
void display(){
    if(top == nullptr){
        cout << "Stack is empty\n";
        return;
    }
    Node*temp = top;
    while(temp != nullptr){
        cout << temp->data << "->";
        temp = temp->next;
    }
    cout << "NULL\n";
}
```

```
};

// main function(from where execution begins)
int main(){
   // Creating an object of Stack class
   Stack stack;

   int value, choice;
   // loop runs till the user wants to exit
   // do-while loop
   do{
      cout << "\nStack Operations:\n";
      cout << "1. Push\n";
      cout << "2. Pop\n";
      cout << "3. Peek\n";
      cout << "4. Check if Empty\n";
      cout << "5. Display Stack\n";
      cout << "6. Exit\n";

      cout << "Enter your choice (1-6) : ";
      cin >> choice;

      // switch-case
      switch(choice){
         case 1:
            cout << "Enter the value to push : ";
            cin >> value;
            stack.push(value);
            break;
```

```
            case 2:
                stack.pop();
                break;

            case 3:
                cout << "Top element is " << stack.peek() << endl;
                break;

            case 4:
                if(stack.isEmpty())
                    cout << "Stack is Empty\n";
                else
                    cout << "Stack is not Empty\n";
                break;

            case 5:
                stack.display();
                break;

            // default case
            default:
                cout << "Invalid Choice. Please try again.\n";
        }
    }while(choice != 6);

    return 0; // indicates no error
}
```

OUTPUT

```
PS C:\Users\junej\CPP> cd "c:\Users\junej\CPP\" ; if ($?) { g++ ImplementStackUsingLinkedList.cpp -o ImplementStackUsingLinkedList } ; if ($?) { .\ImplementStackUsingLinkedList }

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Display Stack
6. Exit
Enter your choice (1-6) : 4
Stack is Empty

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Display Stack
6. Exit
Enter your choice (1-6) : 5
Stack is empty

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Display Stack
6. Exit
Enter your choice (1-6) : 2
Stack underflow
```

```
Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Display Stack
6. Exit
Enter your choice (1-6) : 1
Enter the value to push : 1
1 pushed onto stack

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Display Stack
6. Exit
Enter your choice (1-6) : 1
Enter the value to push : 2
2 pushed onto stack

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Display Stack
6. Exit
Enter your choice (1-6) : 1
Enter the value to push : 3
3 pushed onto stack
```

```
Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Display Stack
6. Exit
Enter your choice (1-6) : 5
3->2->1->NULL

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Display Stack
6. Exit
Enter your choice (1-6) : 3
Top element is 3

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Display Stack
6. Exit
Enter your choice (1-6) : 6
Invalid Choice. Please try again.
PS C:\Users\junej\CPP>
```

Write a program to implement Queue using linked list.

```
// Implement Queue Using Linked List

// HEADER FILES
#include<iostream>
using namespace std;

// Creating a Node structure
struct Node{
    int data;
    Node* next;

    Node(int value) : data(value), next(nullptr) {};
};

// Creating a class Queue Using Linked List
class Queue{
private:
    // There are two pointers in a queue, front and rear and it follows FIFO principle i.e. First In First Out
    Node* front;
    Node* rear;

public:
    // CONSTRUCTOR
    Queue(): front(nullptr), rear(nullptr){}

    // function to insert element in Queue
```

```
void enqueue(int value){
    Node* newNode = new Node(value);

    if(rear == nullptr){
        front = rear = newNode;
        cout << "Enqueued : " << value << "\n";
        return;
    }

    rear->next = newNode;
    rear = newNode;
    cout << "Enqueued : " << value << "\n";
}

// function to remove element from Queue
void dequeue(){
    // empty check
    if(front == nullptr){
        cout << "Queue is empty, cannot dequeue.\n";
        return;
    }

    Node* temp = front;
    front = front->next;

    if(front == nullptr){
        rear = nullptr;
    }
```

```
    cout << "Dequeued : " << temp->data << "\n";
    delete temp;
}

// function to check if queue is empty or not
bool isEmpty(){
    return front == nullptr;
}

// returns peek element in a queue
int peek(){
    if(isEmpty()){
        cout << "Queue is Empty." << endl;
        return -1;
    }
    return front->data;
}

// function to display queue elements
void display(){
    if(front == nullptr){
        cout << "Queue is empty." << endl;
        return;
    }
    Node* temp = front;
    cout << "Queue: ";
    while(temp != nullptr){
        cout << temp->data << " ";
        temp = temp->next;
```

```
        }
        cout << endl;
    }
};

// main function (from where execution begins)
int main(){
    // creating an object for class q
    Queue q;
    int choice, value;

    // do-while loop, runs till the user enters a choice of exit
    do{
        cout << "\nQueue Operations:\n";
        // List of Choices
        cout << "1. Enqueue\n";
        cout << "2. Dequeue\n";
        cout << "3. Peek(Front Element)\n";
        cout << "4. Check if Empty\n";
        cout << "5. Display Queue\n";
        cout << "6. Exit\n";

        // Input choice
        cout << "Enter your choice : ";
        cin >> choice;

        switch(choice){
            case 1:
                cout << "Enter value to enqueue : ";
```

```
        cin >> value;
        q.enqueue(value);
        break;

    case 2:
        q.dequeue();
        break;

    case 3:
        value = q.peek();
        if(value != -1)
            cout << "Front Element : " << value << "\n";
        break;

    case 4:
        cout << "Is queue empty? " << (q.isEmpty() ? "Yes" : "No") << endl;
        break;

    case 5:
        q.display();
        break;

    case 6:
        cout << "Exiting...\n";
        break;

    // default case
    default:
        cout << "Invalid Choice. Please try again.\n";
```

```
    }
  }while(choice != 6);
  return 0; // indicates no error
}
```

OUTPUT

```
PS C:\Users\junej\CPP>
cd "c:\Users\junej\CPP\" ; if ($?) { g++ ImplementQueueFromLinkedList.cpp -o ImplementQueueFromLinkedList } ; if ($?) { .\I
mplementQueueFromLinkedList }

Queue Operations:
1. Enqueue
2. Dequeue
3. Peek(Front Element)
4. Check if Empty
5. Display Queue
6. Exit
Enter your choice : 3
Queue is Empty.

Queue Operations:
1. Enqueue
2. Dequeue
3. Peek(Front Element)
4. Check if Empty
5. Display Queue
6. Exit
Enter your choice : 4
Is queue empty? Yes

Queue Operations:
1. Enqueue
2. Dequeue
3. Peek(Front Element)
4. Check if Empty
5. Display Queue
6. Exit
Enter your choice : 5
Queue is empty.
```

```
Queue Operations:
1. Enqueue
2. Dequeue
3. Peek(Front Element)
4. Check if Empty
5. Display Queue
6. Exit
Enter your choice : 2
Queue is empty, cannot dequeue.

Queue Operations:
1. Enqueue
2. Dequeue
3. Peek(Front Element)
4. Check if Empty
5. Display Queue
6. Exit
Enter your choice : 1
Enter value to enqueue : 1
Enqueued : 1

Queue Operations:
1. Enqueue
2. Dequeue
3. Peek(Front Element)
4. Check if Empty
5. Display Queue
6. Exit
Enter your choice : 1
Enter value to enqueue : 2
Enqueued : 2
```

```
Queue Operations:
1. Enqueue
2. Dequeue
3. Peek(Front Element)
4. Check if Empty
5. Display Queue
6. Exit
Enter your choice : 1
Enter value to enqueue : 3
Enqueued : 3

Queue Operations:
1. Enqueue
2. Dequeue
3. Peek(Front Element)
4. Check if Empty
5. Display Queue
6. Exit
Enter your choice : 5
Queue: 1 2 3

Queue Operations:
1. Enqueue
2. Dequeue
3. Peek(Front Element)
4. Check if Empty
5. Display Queue
6. Exit
Enter your choice : 3
Front Element : 1
```

```
Queue Operations:
1. Enqueue
2. Dequeue
3. Peek(Front Element)
4. Check if Empty
5. Display Queue
6. Exit
Enter your choice : 6
Exiting...
PS C:\Users\junej\CPP>
```

The Celebrity Problem

The Celebrity Problem In a party of N people, only one person is known to everyone. Such a person may be present at the party, if yes, (s)he doesn't know anyone at the party. We can only ask questions like "does A know B? ". Find the stranger (celebrity) in the minimum number of ask questions like "does A know B? ". Find the stranger (celebrity) in the minimum number of questions. We can describe the problem input as an array of numbers/characters representing persons in the party. We also have a hypothetical function Have Acquaintance(A, B) which returns true if A knows B, and false otherwise. How can we solve the problem ?

// The celebrity problem is one in which there will be n persons given in the party

// out of which, one person is a celebrity. We have to identify celebrity among the

// n persons in the party. There will be a given 2D (n X n) matrix having containing

// 0 or 1, where 1 represents a knows b i.e. true and 0 represents a does not b i.e. true

// We also have a hypothetical function HaveAcquaintance(A, B) which returns true

// if A knows B, and false otherwise.

// Two methods to solve the given problem

// FIRST METHOD, Time Complexity : O(n^2)

// we can run a loop on all persons in the party and inside that we can run one more loop

// to check for every other person in the party.

// For celebrity, rows values will be all zero as celebrity does not know anyone

// and the column values will be all 1 except for diagonal elements as every person knows

// the celebrity

// SECOND METHOD, Time Complexity : O(n)

// However, we have to find the celebrity in the minimum number of steps so we should solve

// the question in O(n) time complexity instead of O(n^2) and for that we going to take

// the help of stack

```
// Algorithm to solve the problem using STACKS
// Step 1. insert all the people in the stack(assign them numbers from 0 to n-1 for n persons)
// Step 2. till stack.size() > 1
//       pop two top persons from the stack and check whether if the two persons knows
//       each other or not. Push the one back into stack who don't know the other person
// Step 3. Now only one person is left in the stack, which we call a potential celebrity
//       In this step, confirm that the left person is actually a celebrity or not by
//       checking for the values of have acquitance for the particular row and column
//       Return the celebrity(stranger) if found, else return -1

// HEADER FILES
#include<iostream>
#include<vector>
#include<stack>
using namespace std;

// Returns true if a knows b otherwise false
bool haveAquitance(vector<vector<bool>> &knowsMatrix, int a, int b){
   return knowsMatrix[a][b];
}

// Function to find the celebrity among the group of n people
int celebrity(vector<vector<bool>> &knowsMatrix, int n){
   stack<int> s;
   // STEP 1
   // storing all the celebrity in the stack
   for(int i=0; i<n; i++){
      s.push(i);
   }
```

```
// STEP 2
// Comparing all the values in the stack till only one remains
while(s.size() > 1){
    int a = s.top();
    s.pop();

    int b = s.top();
    s.pop();

    if(haveAquitance(knowsMatrix,a,b)){ // returns true if a knows b
        // push b back as celebrity should know no one
        s.push(b);
    }else{ // executes if a don't know b
        // same case for a
        s.push(a);
    }
}

int potentialCelebrity = s.top();
// STEP 3
// Checking if the potential Candidate is the actual one
bool rowcheck = true;
bool colcheck = true;

for(int i=0; i<n; i++){
    if(knowsMatrix[potentialCelebrity][i] == 1){
        // compare with one as celebrity shouldn't know anyone
        rowcheck = false;
```

```
        }
        if(knowsMatrix[i][potentialCelebrity] == 0 && i != potentialCelebrity){
            // compare with 0 as everone should know celebrity except his own number
            colcheck = false;
        }
    }

    // returns true if both conditions :
    // 1. celebrity knows no other people
    // 2. Every people knows celebrity
    // are satisfied.
    if(rowcheck && colcheck){
        return potentialCelebrity;
    }else{
        return -1;
    }
}

// main function(execution begins from here)
int main(){
    int n; // where n represents the number of people in the party
    cout << "Enter the number of people in the party : ";
    cin >> n;

    vector<vector<bool>> knowsMatrix(n,vector<bool>(n,false));
    // where 0 represents that a does not know b i.e. false and 1 represents true or a knows b
    cout << "Enter the acquitance matrix(values 0 or 1)\n";
    // Input knows Matrix
    for(int i=0; i<n; i++){
```

```
        for(int j=0; j<n; j++){
            int knows;
            cin >> knows;
            knowsMatrix[i][j] = knows;
        }
    }

    // Output celebrity, -1 if celebrity does not exist
    cout << "The celebrity in the party is : " << celebrity(knowsMatrix, n) << endl;

    return 0; // indicates no error
}
```

OUTPUT

OUTPUT 1 - Celebrity exists among n people

```
PS C:\Users\junej\CPP> cd "c:\Users\junej\CPP\" ; if ($?) { g++ TheCelebrityProblem.cpp -o TheCelebrityProblem } ; if ($?)
{ .\TheCelebrityProblem }
Enter the number of people in the party : 4
Enter the acquitance matrix(values 0 or 1)
0 1 1 0
1 0 1 1
0 0 0 0
1 1 1 0
The celebrity in the party is : 2
```

OUTPUT 2 – Celebrity does not exist among n people

```
PS C:\Users\junej\CPP> cd "c:\Users\junej\CPP\" ; if ($?) { g++ TheCelebrityProblem.cpp -o TheCelebrityProblem } ; if ($?)
{ .\TheCelebrityProblem }cd "c:\Users\junej\CPP\" ; if ($?) { g++ TheCelebrityProblem.cpp -o TheCelebrityProblem } ; if ($?
) { .\TheCelebrityProblem }
Enter the number of people in the party : 3
Enter the acquitance matrix(values 0 or 1)
0 1 1
1 0 1
1 1 0
The celebrity in the party is : -1
Enter the number of people in the party :
```